

Ứng dụng của thứ tự

Long Fan
Trường Trung học Yali, Trường Sa

2005

Nguồn gốc: Ứng dụng của thứ tự

IOI China candidate team papers / 2005 / Long Fan.doc

Tóm tắt

Bản chất của thi tin học là khai phá dữ liệu, còn “thứ tự” là một quan hệ thường gặp nhưng khó phát hiện ẩn giữa dữ liệu. Nếu trong quá trình giải bài ta tìm được quan hệ thứ tự ẩn trong đề, bài toán thường trở nên sáng rõ. Bài viết tập trung thảo luận cách phát hiện và xây dựng thứ tự thích hợp trong các quan hệ dữ liệu phức tạp, từ đó làm đơn giản và giải quyết bài toán.

Từ khóa: quan hệ dữ liệu; thứ tự; cây; dãy DFS; đồ thị; thứ tự tô-pô.

1 Dẫn nhập

Thứ tự được dùng rất rộng trong tin học. Những ví dụ cơ bản nhất là tìm kiếm nhị phân dựa trên thứ tự lớn nhỏ, và quy hoạch động trên đồ thị có hướng không chu trình dựa trên thứ tự tô-pô. Để có những thứ tự ấy, ta có các thuật toán tương ứng như sắp xếp nhanh và sắp xếp tô-pô.

Đồ thị, cây, dãy tuyến tính, tập hợp, v.v. là những quan hệ dữ liệu khác nhau, và mỗi loại quan hệ có những thứ tự tương ứng. Ngay cả cùng một quan hệ dữ liệu cũng có nhiều thứ tự khác nhau tùy theo ý nghĩa đang xét. Chẳng hạn, đồ thị có hướng có thứ tự DFS theo nghĩa duyệt sâu, nhưng có thứ tự tô-pô theo nghĩa phụ thuộc trước sau. Bản thân thứ tự không nhất thiết phải là tuyến tính; thứ tự tô-pô cũng không phải một thứ tự tuyến tính chặt.

Trong dữ liệu rối rắm, tìm được thứ tự có giá trị và dùng nó hợp lý chính là chủ đề của bài viết này.

2 Dùng thứ tự để giải bài toán

Nhiều bài tương tác sẽ trở nên dễ hơn nếu tìm được thứ tự thích hợp rồi khai thác tính chất của nó. Xét ví dụ sau.

2.1 Bài toán lưới

Mô tả bài toán. Có một lưới điểm $N \times N$, với

$$1 \leq N \leq 2003.$$

Giữa hai điểm kề nhau có một cung có hướng mang trọng số nguyên w , $1 \leq w \leq 500000$. Đồng thời, từ điểm góc trên trái $(1, 1)$ tới cùng một điểm bất kỳ, mọi đường đi đều có cùng độ dài, tính bằng tổng trọng số các cung đi qua.

Mỗi lần ta có thể hỏi hai giá trị liên quan tới một điểm (x, y) : khoảng cách từ điểm đó tới điểm ngoài cùng bên phải cùng hàng, và khoảng cách từ điểm đó tới điểm dưới cùng cùng cột. Cần tìm một điểm sao cho khoảng cách từ góc trên trái tới điểm ấy bằng số nguyên cho trước L ,

$$1 \leq L \leq 2 \cdot 10^9.$$

Ban đầu chỉ biết N và L , và được hỏi nhiều nhất 6667 lần.

Phân tích. Trước hết có thể phủ định ý tưởng đơn giản nhất: hỏi để tính ra độ dài từng cạnh rồi giải hệ phương trình. Tổng số cạnh là $2N(N - 1)$. Từ điều kiện mọi đường đi từ $(1, 1)$ tới cùng một điểm có độ dài như nhau, ta chỉ có thể suy ra khoảng $(N - 1)^2$ phương trình độc lập, nên vẫn cần quá nhiều câu hỏi. Khi $N = 2003$, giới hạn 6667 chỉ xấp xỉ $3N$, vì thế hướng này không thể dùng.

Hãy nhìn lại yêu cầu của đề: ta chỉ cần tìm một điểm có khoảng cách từ góc trên trái bằng L . Ta không quan tâm trọng số từng cạnh, mà chỉ quan tâm khoảng cách của từng điểm tới góc trên trái. Gọi giá trị đó là trọng số của điểm.

Một sự thật quan trọng là: trọng số của một điểm luôn nhỏ hơn trọng số của mọi điểm nằm phía dưới bên phải nó. Đây là thứ tự cơ bản nhất của bài toán. Nó không dễ dùng trực tiếp, nhưng gợi ý rằng nếu trọng số tăng dần từ trái trên sang phải dưới, thì trên các đường chéo từ trái dưới lên phải trên, giá trị có thể được khai thác theo một cách khác.

Ý tưởng là tìm trước một điểm ở phía trái dưới có trọng số gần L , rồi đi dần theo hướng phải trên. Cụ thể:

1. Trên cột ngoài cùng bên trái, dùng tìm kiếm nhị phân để tìm điểm có trọng số vừa nhỏ hơn L , còn điểm ngay dưới nó đã lớn hơn L .
2. Từ điểm đó, trên cùng hàng, dùng tìm kiếm nhị phân để tìm điểm gần L nhất. Gọi điểm bắt đầu kiểm tra là S .

Theo cách chọn S và thứ tự cơ bản ở trên, các vùng bên trái, bên dưới, và một phần bên phải dưới của S đều không thể chứa nghiệm. Nghiệm nếu tồn tại chỉ còn nằm trong vùng phía phải trên của điểm đang kiểm tra.

Khi xét điểm hiện tại:

- Nếu trọng số bằng L , ta đã tìm được nghiệm.
- Nếu trọng số lớn hơn L , mọi điểm cùng hàng ở bên phải càng lớn hơn, nên không thể là nghiệm; ta dịch điểm kiểm tra lên trên.
- Nếu trọng số nhỏ hơn L , mọi điểm cùng cột ở phía trên không phù hợp theo vùng còn lại; ta dịch điểm kiểm tra sang phải.

Ta luôn giữ bất biến rằng nghiệm chỉ có thể nằm trong vùng phía phải trên của điểm đang kiểm tra. Mỗi bước thu hẹp vùng ấy bằng cách đi lên hoặc sang phải. Hai lần tìm kiếm nhị phân ban đầu cộng với quá trình đi đơn điệu này cần số câu hỏi tuyến tính theo N , nhỏ hơn giới hạn khi N đạt cực đại.

Từ một thứ tự rất đơn giản và cơ bản, cộng với việc thiết kế thuật toán phù hợp với đặc điểm của nó, ta đã giải được bài toán. Điều này nhắc ta không nên xem nhẹ những tính chất nội tại có vẻ đơn giản trong đề bài.

3 Dùng thứ tự để đơn giản hóa bài toán

3.1 Bài toán dựng cây

Mô tả bài toán. Cho một cây có n đỉnh, đánh số $1, 2, \dots, n$. Mỗi đỉnh v có một trọng số $s(v)$, cũng là một hoán vị của $1, 2, \dots, n$. Với đỉnh v , định nghĩa $t(v)$ là số hậu duệ của v có trọng số nhỏ hơn $s(v)$. Biết cây và các giá trị

$$t(1), t(2), \dots, t(n),$$

hãy tìm một cách gán trọng số cho các đỉnh. Nếu có nhiều nghiệm, in một nghiệm bất kỳ.

Phân tích. Thoạt nhìn bài toán khó bắt đầu, nhưng chỉ cần nắm định nghĩa của $t(v)$ thì cách giải trở nên nhẹ hơn. Định nghĩa ấy nói tới các hậu duệ có trọng số nhỏ hơn $s(v)$, nên gợi ý dùng thứ tự duyệt trước DFS. Một tính chất quan trọng của duyệt trước là: các hậu duệ của một đỉnh xuất hiện ngay sau đỉnh ấy trong dãy duyệt, dưới dạng các đoạn con liên tiếp của các cây con.

Hãy tưởng tượng có n ô trống từ trái sang phải, tương ứng với các trọng số $1, 2, \dots, n$. Nếu ô thứ j được điền đỉnh i , ta hiểu rằng $s(i) = j$. Theo thứ tự duyệt trước DFS, lần lượt đặt từng đỉnh vào các ô. Khi xử lý đỉnh i , ta đặt nó vào ô trống thứ $t(i) + 1$ tính từ trái sang. Lý do là sau nó còn có đúng $t(i)$ hậu duệ phải mang trọng số nhỏ hơn nó, tức phải được đặt vào các ô trống bên trái nó. Nếu lần nào cũng chừa đúng $t(i)$ ô trống bên trái, điều kiện của bài được thỏa mãn.

Chứng minh phác thảo. Trước hết chứng minh bằng quy nạp rằng với một cây con có N đỉnh, thuật toán sẽ đặt toàn bộ đỉnh của cây con vào N ô trống đầu tiên đang dành cho cây con đó. Với $N = 1$ hiển nhiên đúng vì $t(v) = 0$. Với $N = k$, gọi gốc là r và các cây con của nó lần lượt là $S_1, S_2, \dots, S_m$. Dãy duyệt trước có dạng

$$r, S_1, S_2, \dots, S_m.$$

Sau khi đặt r vào ô trống thứ $t(r) + 1$, từng cây con có kích thước nhỏ hơn k được xử lý theo giả thiết quy nạp, nên toàn bộ cây con của r cùng với r chiếm đúng các ô trống đầu tiên dành cho cây ấy.

Tiếp theo, với mỗi đỉnh i , các ô trống bên trái được chừa lại trước khi đặt i đều sẽ được lấp bởi các hậu duệ của chính nó. Điều này suy ra từ nhận xét trên: đỉnh i và các hậu duệ của nó chiếm một đoạn ô đầu tiên trong phạm vi cây con của i .

Như vậy bài toán được đưa về dạng: có một dãy N ô trống, mỗi lần nhận lệnh “đặt số A_i vào ô trống thứ P_i ”, cần tìm trạng thái cuối cùng. Bài toán này có thể giải bằng cây Fenwick hoặc cây đoạn: ghi nhận ô nào còn trống, và mỗi lần tìm ô trống thứ P_i trong $\mathcal{O}(\log N)$. Tổng độ phức tạp là

$$\mathcal{O}(N \log N),$$

phần DFS chỉ mất $\mathcal{O}(N)$, còn bộ nhớ là $\mathcal{O}(N)$.

3.2 Bài toán xếp hàng binh sĩ

Mô tả bài toán. Có N binh sĩ và một hàng vị trí đánh số từ trái sang phải. Mỗi lệnh của tướng có dạng $\text{Goto}(L, S)$.

- Nếu vị trí L đang trống, binh sĩ S đứng vào vị trí L .
- Nếu vị trí L đang có binh sĩ K , thì S đứng vào vị trí L , còn K bị đẩy sang bằng cách thực hiện tiếp $\text{Goto}(L+1, K)$.

Tướng ban ra N lệnh; mỗi binh sĩ được nhắc tới đúng một lần. Cần tìm trạng thái hàng cuối cùng. Trong quá trình thực hiện, vị trí binh sĩ đứng có thể vượt quá giới hạn ban đầu, nên trước hết cần xác định độ dài cuối cùng của hàng; ô trống được in là 0.

Phân tích ban đầu. Mô phỏng trực tiếp có thể mất $\mathcal{O}(N^2)$ vì một lần chen gây ra dây chuyền đẩy dài. Có thể tối ưu mã bằng cấu trúc hợp nhất tập hợp và thao tác dịch khối, nhưng về bản chất độ phức tạp xấu nhất vẫn chưa thay đổi.

Một cách mô phỏng khác là xét cả khối liên tiếp. Một khối là một đoạn binh sĩ liên nhau. Khối được xem như một chỉnh thể đối với bên ngoài, còn bên trong lưu vị trí tương đối của các thành viên. Vì cần tìm kiếm và chen phần tử trong khối, ta dễ nghĩ tới cây cân bằng, trong đó khóa của mỗi phần tử là vị trí tương đối bên trong khối. Tuy nhiên lại xuất hiện vấn đề gộp khối: nếu khoảng cách giữa hai khối trở thành 0, chúng phải hợp nhất. Cây cân bằng không dễ hợp nhất; cách thô là tháo từng phần tử của khối nhỏ để chen vào khối lớn. Phân tích khấu hao cho thời gian xấu nhất khoảng $\mathcal{O}(N \log^2 N)$, và cài đặt khá rườm rà. Một cách dùng cây cân bằng tốt hơn là xây dựng cây trên toàn bộ các ô theo thứ tự, mỗi nút ghi số ô trống để định vị và xóa; cách này đạt $\mathcal{O}(N \log N)$.

Bài toán còn có một cách đơn giản và hiệu quả hơn. Điểm then chốt là thao tác chen gây ra dây chuyền đẩy. Xét ví dụ có hai lệnh

Goto(2,1), Goto(2,2).

Khi thực hiện lệnh thứ hai, binh sĩ 1 bị đẩy. Nhưng nếu ta điền binh sĩ 2 trước, rồi điền binh sĩ 1 vào ô trống thứ hai, hiệu quả cuối cùng là tương đương. Điều này giống hệt bài toán dựng cây sau khi chuyển về dạng điền số vào ô trống.

Định nghĩa lệnh mới newgoto(L, S): đặt binh sĩ S vào ô trống thứ L , trong đó L là thứ tự trong các ô trống chứ không phải chỉ số vị trí tuyệt đối. Câu hỏi trở thành: với một dãy lệnh Goto, có tồn tại một dãy lệnh newgoto tương đương chỉ bằng cách đổi thứ tự các cặp (L, S) hay không?

Câu trả lời là có. Thứ tự cần tìm phải thỏa hai điều kiện:

1. Nếu hai khối không liên quan A, B và A nằm bên trái B , thì mọi phần tử của B phải xuất hiện trước mọi phần tử của A trong dãy newgoto. Nếu chen A trước, nó sẽ làm vị trí của B dịch sang phải.
2. Nếu việc chen binh sĩ S làm binh sĩ K bị đẩy, thì S phải được chen trước K trong dãy newgoto. Đây chính là nguyên lý tránh dây chuyền đẩy.

Nếu xem mỗi binh sĩ là một đỉnh, và đặt cạnh có hướng $A \rightarrow B$ khi A phải được chen trước B , thì thứ tự cần tìm là một thứ tự tô-pô của đồ thị này.

Ta không thể xây dựng đồ thị rồi sắp xếp tô-pô trực tiếp: việc tìm từng cạnh gần như đã là mô phỏng lệnh Goto, và số cạnh có thể là bậc hai. Thay vào đó, dùng mô phỏng không hoàn chỉnh: khi mô phỏng, không lưu quan hệ vị trí bên trong khối, vì quan hệ ấy không ảnh hưởng tới thứ tự cần tìm.

Với mỗi khối, dùng cấu trúc hợp nhất tập hợp để lưu thông tin cơ bản của khối và một dãy lệnh newgoto riêng của khối, ký hiệu $\text{cmd}(A)$. Ban đầu mọi vị trí đều trống. Khi xử lý lệnh Goto(L_i, i):

Chèn vào ô trống. Nếu i được chen vào một vị trí trống, ta tạo khối mới. Dãy lệnh của khối này chỉ gồm chính cặp lệnh

$$\text{cmd}(i) = (L_i, i).$$

Chèn vào một khối. Nếu i được chen vào một khối A , ta đặt i vào khối và cập nhật dãy lệnh bằng cách đưa cặp (L_i, i) lên đầu:

$$\text{cmd}(A + i) = (L_i, i) + \text{cmd}(A).$$

Phần bị đẩy do i chen vào phải xuất hiện sau i ; phần không bị đẩy cũng phải xuất hiện sau i để vị trí của i không bị ảnh hưởng. Vì vậy đưa (L_i, i) lên đầu là hợp lý.

Hợp nhất khối. Một lần chen có thể làm hai khối liền nhau và cần hợp nhất. Nếu khối A nằm bên trái khối B , thì mọi phần tử của B phải xuất hiện trước mọi phần tử của A trong dãy newgoto. Do đó

$$\text{cmd}(A + B) = \text{cmd}(B) + \text{cmd}(A).$$

Các trường hợp hợp nhất thường gặp đều xử lý theo nguyên tắc này.

Sau khi mô phỏng hết, lấy các khối còn lại theo thứ tự từ phải sang trái và nối các dãy cmd của chúng lại. Dãy thu được chính là thứ tự tô-pô cần tìm, và dãy newgoto này tương đương với dãy Goto ban đầu.

Quá trình biến đổi lệnh có thể làm gần tuyến tính bằng hợp nhất tập hợp và danh sách nối. Sau đó mô phỏng dãy newgoto bằng cây Fenwick hoặc cây đoạn, giống bài dựng cây, trong $\mathcal{O}(N \log N)$. Tổng độ phức tạp là

$$\mathcal{O}(N \log N).$$

Qua hai bài toán trên, ta đã dùng những thứ tự khác nhau để đưa các bài tưởng như không liên quan về cùng một dạng: điền số vào ô trống thứ k , giải bằng cây Fenwick hoặc cây đoạn. Thứ tự thích hợp giúp ta thấy bản chất của bài toán rõ hơn.

4 Kết luận

Những bài toán khác nhau ẩn chứa những thứ tự khác nhau để ta khai phá. Nếu biết tận dụng, thứ tự sẽ như một đôi mắt giúp ta nhìn thấu bài toán hơn, và nhìn rõ bản chất trong những quan hệ dữ liệu rối rắm.

Các ví dụ trên cũng cho thấy việc dùng thứ tự không hề đơn giản. Có những thứ tự ẩn rất sâu, thậm chí phải tự xây dựng mới có. Điều này đòi hỏi tích lũy thường xuyên và sự nhạy cảm với các quan hệ tinh tế giữa dữ liệu.

Lời cảm ơn

Xin cảm ơn bạn Lei Tao vì bài gốc tự sáng tác và bạn Li Shi vì phần cải biên đề bài. Xin cảm ơn Ren Kai và Zhou Yuan đã giúp đỡ trong bài toán xếp hàng binh sĩ.

Tài liệu tham khảo

- Đề thi CEOI 2003.
- *Introduction to Algorithms*.
- Đề gốc do Lei Tao, Trường Trung học Yali, Trường Sa, Hồ Nam, sáng tác.
- Luận văn đội tuyển tập huấn quốc gia năm 2002 của Li Rui.